

COMP.2560: System Programming Assignment #3: Process Control

ASSIGNMENT 3: Overview

The purpose of this assignment is to evaluate your comprehension on Process Control.

There are three (3) questions in this assignment, described below.

ASSIGNMENT 3 – Question 1

The code in file “`abort.c`” (posted on the course website) uses hard-coded bit- manipulation to access the child process termination status.

Rewrite this code using the bit-manipulation macros discussed in the lecture.

ASSIGNMENT 3 – Question 2

Using Unix system calls, `fork()`, `wait()`, `read()` and `write()`, write a C program for integer- basic Arithmetic Calculations (+, -, *, /).

Your program should follow the sequential steps, given below:

1. Prompts the message “This program makes simple arithmetic calculations.”,
2. Gets in an infinite loop.
Then:
 1. Writes the message “Enter an arithmetic statement, e.g., `34 + 132 >` ”,
 2. Reads the whole input line,
 3. Forks, and
 - a. The parent writes the message “Created a child to make your operation, waiting ...”, then
 - b. Calls `wait()` to wait for its child.
 - c. The child process calls the function `childFunction(char *)` and never returns.
 4. The child, through `childFunction(char *line)`:
 - a. Writes the message “I am a child working for my parent.”,
 - b. Uses `sscanf()` to convert the input line into an integer, a character and an integer, respectively,
 - c. In case of a wrong statement, the child calls `exit(50)` ,
 - d. In case of division by zero, the child calls `exit(100)` ,
 - e. In case of a wrong op the child calls `exit(200)` ,
 - f. Otherwise, it performs the appropriate arithmetic operation,
 - g. Uses `sprint()` to create an output buffer consisting of “`n1 op n2 = result`”,
 - h. Writes the output buffer to the screen,
 - i. Calls `exit(0)` ,

5. Once the child terminates, the parent checks the returned status value using bit-manipulation macros discussed in class and if it is 50, 100 or 200, writes “Wrong statement”, “Division by zero” or “Wrong operator”, respectively.
6. The parent goes back to step 1.

Important notice:

- All reads/writes must be done using `read()/write()`, even if it is to write to the console or read from the keyboard, unless otherwise specified.
- You can use the returned value of `sscanf()` for detecting a “Wrong statement”¹.

ASSIGNMENT 3 – Question 3

Write a C program to execute multiple Unix commands in parallel, following the following requirements:

- The number of Unix commands to be executed is not fixed.
- There is no communication among the Unix commands.
- The Unix commands are given as command line arguments.
- For simplicity, you can assume that each Unix command has exactly one argument except that the last one can have either no argument or one argument.

For example:

> `miniminishell cat openfile.c ls -t ps` includes three Unix commands:

`cat`, with one argument (`openfile.c`),

`ls`, with one argument (`-t`), and

`ps`, with no argument.

- For each Unix command, use a separate child process to execute it. You need to print out each process’s id.
- Observe how the output from each child process is displayed.
- Are the outputs from all the child processes displayed in a certain fixed order or intermingled?

Note: In case you are interested and thinking about how multiple processes share the terminal window, you can refer to Chapter 9.5. and Chapter 9.6 (not required by this course though).

ASSIGNMENT 3 – Wrap-up and Submission

- For all questions, you need to submit your source code.
Also, use the script command with the timing option to record you compiling and running your program and submit the script file and timing file for each question as well.
Submit the source code for each question and include any accompanying files (if applicable) needed for that question.
Please make sure all your code works properly on CS Linux server.
- Submit the link to the video you will create.

¹ **Hint:** Review the sample code in file “`ReadInt.c`” to see how to use `sscanf(...)`, or google online to see how to use `sscanf()`.



-
- Please properly name all your files so that markers can easily identify which file is for which question. See the *Assignment Submission Specification document* for naming files in your submissions for this semester.
 - Record one video with audio showing your source code, you compiling and running each program and explaining how each of the underlined items in each question's description is realized in your code. For the convenience of marking your video submission, please add chapters to your video so that markers can easily jump to the start of each question in your video. If you do not know how to add chapters to your video, please watch this short video to learn how:
 - <https://www.youtube.com/watch?v=d5uSWaCPLj4>
- If you are using Microsoft Streams, this link shows how to add chapters:
- <https://support.microsoft.com/en-us/office/using-manual-chapters-with-videos-on-stream-on-sharepoint-8bbf61eb-c00b-42b5-a514-cce2e45eb6ea>
- Please note the late assignment submission policy stated in the course syllabus.